Software Testing

What is st:

It is a method to check whether the actual sw product matches expected requirements and to ensure that sw product is defect free.

It involves execution of sw/system components using manual or automated tools to evaluate one or more properties of interest.

The purpose of sw testing is to identify errors, gaps or missing requirements in contrast to actual requirements.

Why we need st:

To ensure sw quality

To find and fix defects early

To improve security and detect vulnerability

To ensure reliability and stability

To validate business and user requirements

To enhance UI

To build customer trust and confidence

To reduce long term maintenance costs

To ensure compatibility across devices, browsers, OS, and env.

To prevent costly failures in production

To verify performance under load and stress

To support continuous improvement through feedback.

What if st is not there:

More bugs will reach production

Higher cost to fix issues later

Increased security vulnerability

Poor UX

Loss of customer trust

Frequent system crashes or failures

Delays due to rework

Risk of business losses

Non compliance with industry standards

Poor performance.

Alternatives of st:

Formal verification

Code reviews / pair programming

Linters and formatters

Strongly typed lang

When to use st:

When gathering and validating req

When designing system architecture and workflows

During dev

After implementing new features

Before releasing the product

After deployment for monitoring and validation

When fixing bugs or modifying existing features

When intergrating third party APIs or services

Scenarios st:

Login and authentication

E-commerce checkout

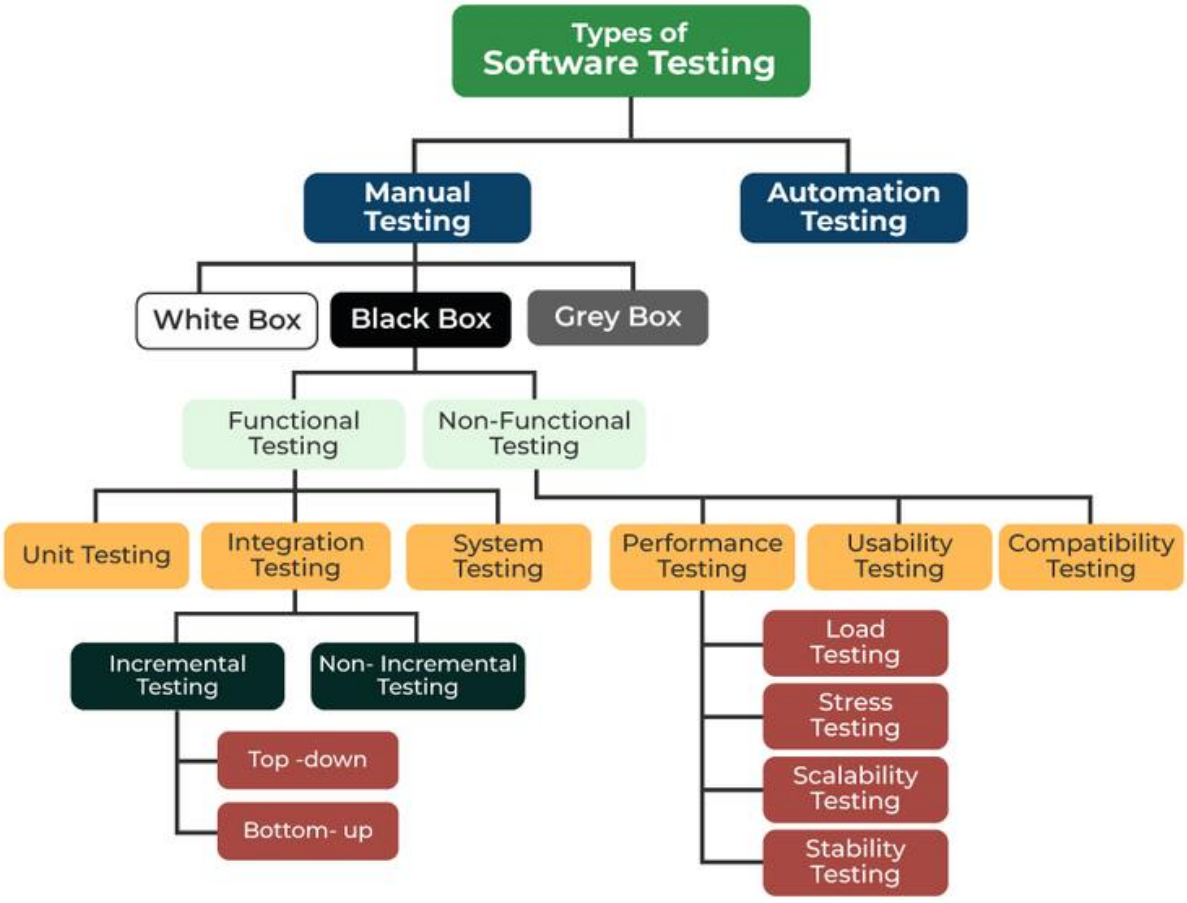
Form submission

File uploads and downloads

Api integration

Database testing

Types of software testing:



Software testing types:

Black box testing:

It is a sw testing method in which the internal structure, design , implementation of the item being tested is not known to the tester.

Only the external design and structure are tested.

Types:

functional testing

non functional testing

Functional testing Vs non functional testing:

Criteria	Functional Testing	Non-Functional Testing
Purpose	Evaluates the functionality of the software application and ensures it meets specified requirements.	Evaluates the non-functional aspects of the software application, such as performance, usability, security, and reliability.
Testing Techniques	Black box testing, White box testing, Unit testing, Integration testing, System testing, Acceptance testing.	Load testing, Stress testing, Usability testing, Security testing.
Automation	Often performed manually, but can be automated using frameworks like Selenium or Appium.	Typically automated using tools like Apache JMeter, LoadRunner, or SoapUI.
Metrics	Number of features tested, Pass/Fail rate.	Transaction success rate, Response time, Resource utilization rate.

White box testing:

It is a sw testing method in which the internal structure, design, implementation of a item being tested is known to the tester .

Implementation and impact of the code are tested.

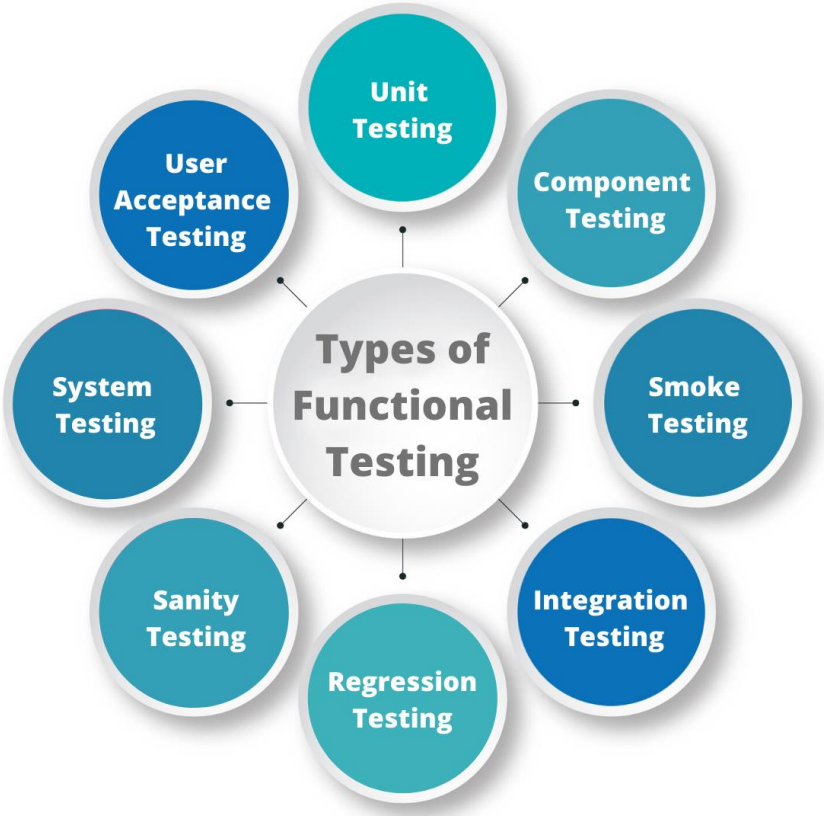
Path testing:

It is individual units or components of a sw are tested. The purpose is to validate that each unit of the sw code performs as expected.

Component testing:

Component testing also known as program testing or module testing is a tykpe of sw testing that is done after the unit testing.

Functional testing:



Unit testing:

It is individual unit or components of sw are tested. The purpose is to validate that each unit of sw code performs as expected.

Developer doing the unit testing – **MCQ**

Component testing:

It is also known as program testing or module testing is a type of sw

Smoke testing:

It si also called build verification testing or confidence testing, is sw testing method that is used to determine if a new sw build is ready for the next testing phase.

This testing method determines if the most crucial functions of program work but does not delve into finer details.

Integration testing:

It is defined as type of testing where sw modules are integrated logically and tested as a group.

A typical sw project consists of multiple sw modules, coded by different programmers.

System testing:

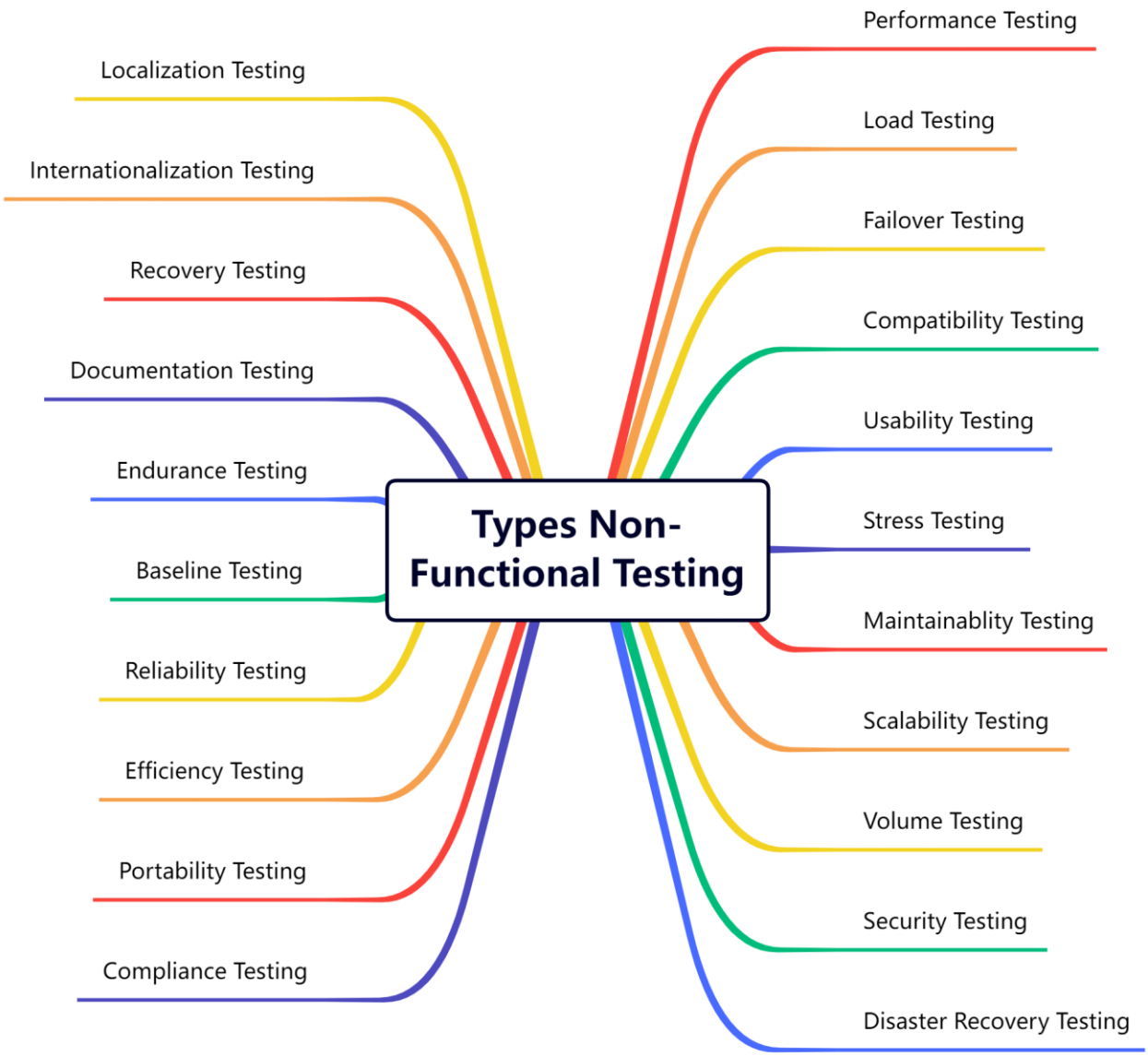
It is a type of sw testing that is performed on the complete integrated system to evaluate the compliance of the system with the corresponding requirements

User acceptance testing:

It is often the last phase of st process and is performed before the tested sw is released to its intended market.

Non functional testing:

Types:



performance testing:

It is a type of sw testing that ensures sw app perform properly under their expected workload.

It is a testing technique carried out to determine system performance in terms of sensitivity, reactivity, and stability under a particular workload.

Security testing:

Security scanning is the identification of network and system weakness.

Usability testing:

It is a type of testing to ensure that a sw program or system is ease to use.

Compatibility testing:

It is a type of testing to ensure that a sw program or system is compatible with other sw programs or systems.

What is test management:

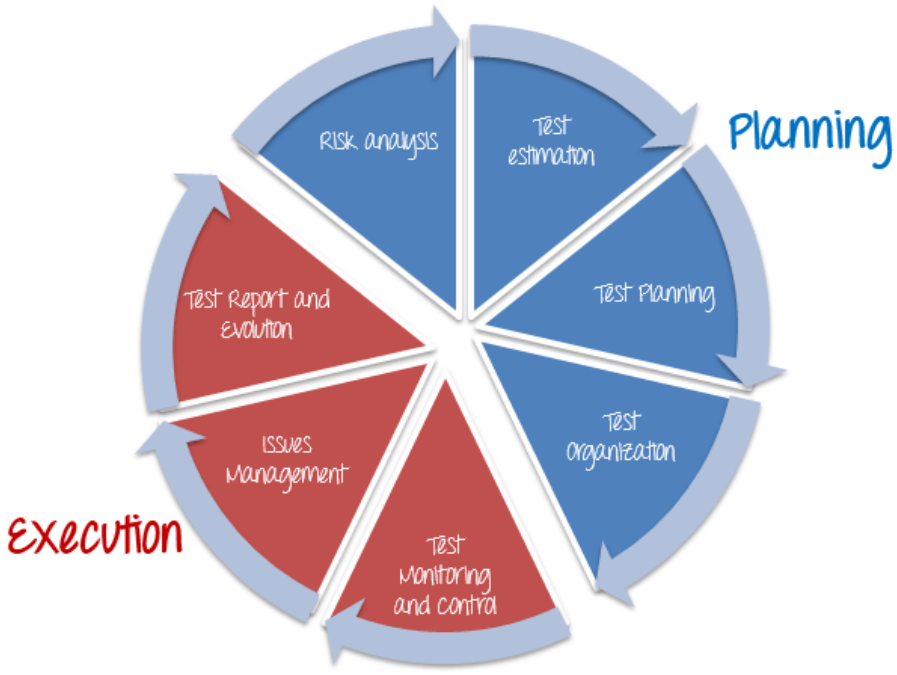
It is a process of managing the testing activities in order to ensure high quality and high end testing of sw app.

The method consists of roganizing, controlling, ensuring traceability and visibility of the testing process in order to deliver.

2 main parts:

planning

execution



Planning:

Risk analysis:

All project may contain risk, early risk detection and identification of its solution will help test manager to avoid potential loss in the future and save on project costs.

Test estimation:y

It is approximately determining how logn a task would take to complete.

Test planning:

It can be defined as a document describing a scope, approach, resources, and schedule of intended testing activities.

Test organization:

It defines who is responsible for which activities in test process.

Execution:

Test monitoring and control:

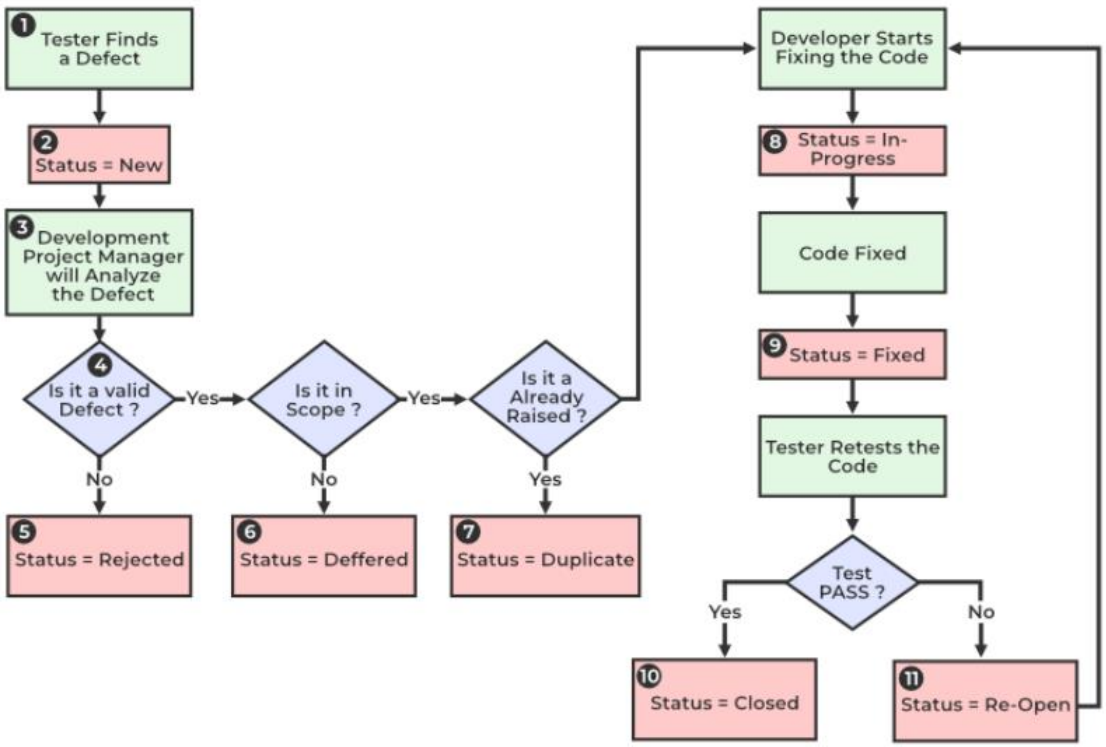
It is a process of overseeing all the metrics necessary to ensure that the project is running well, on schedule , and not out of budget.

Bug life cycle:

Comparison basis	Bug	Defect	Error	Fault	Failure
Definition	It is an informal name specified to the defect.	The **Defect** is the difference between the actual outcomes and expected outputs.	An **Error** is a mistake made in the code; that's why we cannot execute or compile code.	The **Fault** is a state that causes the software to fail to accomplish its essential function.	If the software has lots of defects, it leads to failure or causes failure.
Raised by	The **Test Engineers** submit the bug.	The **Testers** identify the defect. And it was also solved by the developer in the development phase or stage.	The **Developers and automation test engineers** raise the error.	**Human mistakes** cause fault.	The failure finds by the manual test engineer through the **development cycle.**

Different types	Different type of bugs are as follows: ◦ Logic bugs ◦ Algorithmic bugs ◦ Resource bugs	Different type of Defects are as follows: Based on **priority**: ◦ High ◦ Medium ◦ Low And based on the severity: ◦ Critical ◦ Major ◦ Minor ◦ Trivial	Different type of Error is as below: ◦ Syntactic Error ◦ User interface error ◦ Flow control error ◦ Error handling error ◦ Calculation error ◦ Hardware error ◦ Testing Error	Different type of Fault are as follows: ◦ Business Logic Faults ◦ Functional and Logical Faults ◦ Faulty GUI ◦ Performance Faults ◦ Security Faults ◦ Software/ hardware fault	-----
Reasons behind	Following are reasons which may cause the **bugs:** Missing coding Wrong coding Extra coding	The below reason leads to the **defects**: Giving incorrect and wrong inputs. Dilemmas and errors in the outside behavior and inside structure and design. An error in coding or logic affects the software and causes it to breakdown or the failure.	The reasons for having an **error** are as follows: Errors in the code. The Mistake of some values. If a developer is unable **to compile or run a program successfully**. Confusions and issues in programming. Invalid login, loop, and syntax. Inconsistency between actual and expected outcomes. Blunders in design or requirement actions. Misperception in understanding the requirements of the application.	The reasons behind the **fault** are as follows: A Fault may occur by an improper step in the initial stage, process, or data definition. Inconsistency or issue in the program. An irregularity or loophole in the software that leads the software to perform improperly.	Following are some of the most important reasons behind the **failure:** Environmental condition System usage Users Human error
Way to prevent the reasons	Following are the way to stop the **bugs**: Test-driven development. Offer programming language support. Adjusting, advanced, and operative development procedures. Evaluating the code systematically.	With the help of the following, we can prevent the **Defects**: Implementing several innovative programming methods. Use of primary and correct software development techniques. Peer review It is executing consistent code reviews to evaluate its quality and correctness.	Below are ways to prevent the **Errors**: Enhance the software quality with system review and programming. Detect the issues and prepare a suitable mitigation plan. Validate the fixes and verify their quality and precision.	The **fault** can be prevented with the help of the following: Peer review. Assess the functional necessities of the software. Execute the detailed code analysis. Verify the correctness of software design and programming.	The way to prevent **failure** are as follows: Confirm re-testing. Review the requirements and revisit the specifications. Implement current protective techniques. Categorize and evaluate errors and issues.

Bug life cycle: - **MCQ**



Different states:

New: when a new defect is logged

Assigned: once the bug is posted by tester

Open: teh developer start analyzing

Fixed: made necessary changes

Retest: tester does the retesting of the code at this state to check whether the defect is fixed by the developer or not and changes the status to re-test

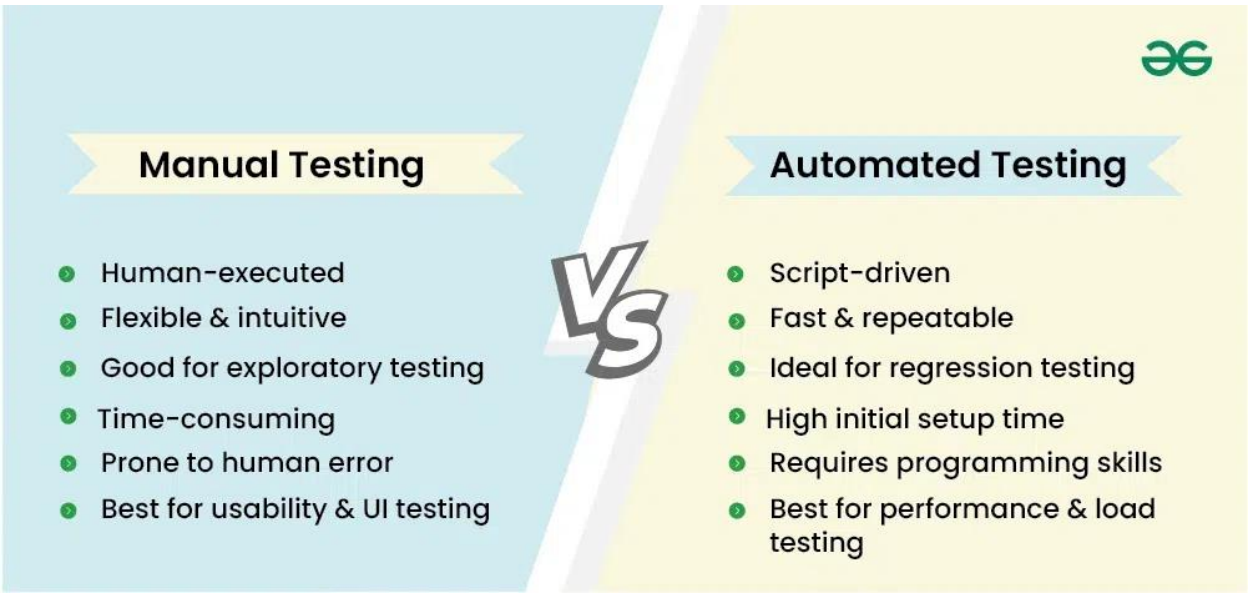
Pending retest:

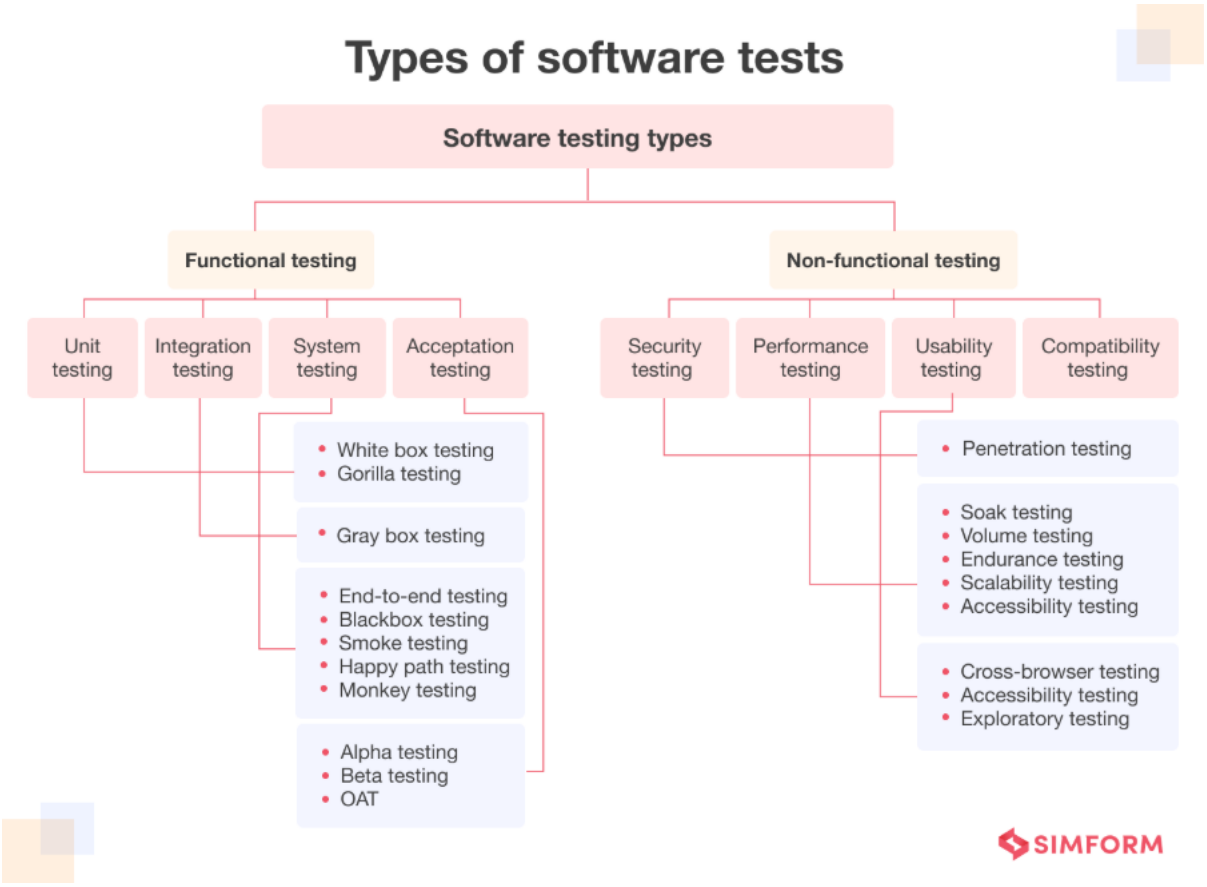
Rejected

Deferred

Not a bug

Manual testing vs automation testing:





What is manual testing:

It is a sw testing process in which test cases are executed manually without using any automated tool.

All test cases executed by the tester mannually according to the end user's perspective.

Why we need Manual Testing:

Human intuition can detect issues automation misses.

Helps test early stage features before automation is possible

Essential for exploratory and ad hoc testing

Humans can evaluate UI/UX better than scripts

Required where automation struggles (captcha, hardware, dynamic UI)

When to use Manual Testing:

Exploratory testing
    ad-hoc testing

Usability testing
    user acceptance testing

Testing new features

Short term or one time testing

Visual / UI testing

Scenarios requiring human judgment

Scenarios of Manual Testing:

Usability / Ui testing

How to perform Manual Testing:

Step1: first, review all doc related to sw, for selecting the testing areas.

Step2: analyze all the required doc to get all the req mentioned by the end user.

Step3: build test cases as per the requirement doc

Step4: execute all the test cases manually by using white box testing and black box testing

Step5:

Types of Manual Testing:

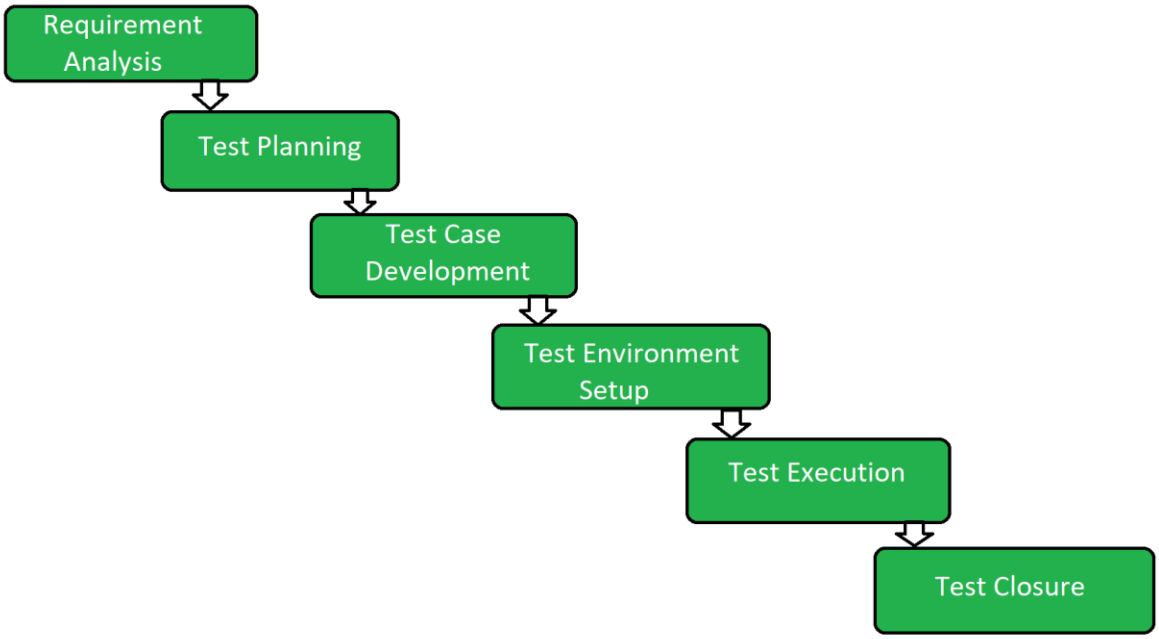
- White box testing – tests by the developer
- Black box testing – no internal implementation, tests by test engineer
- Grey box testing – know both coding and testing

Roles and responsibility of manual tester:

- Gathering and analyzing the client requirements
- Establishing the test env for executing the test cases
- Analyzing and running the test cases
- Arranging review meetings
- Communicating with the users to understand the product requirements
- Interacting with the test manager
- Defect tracking
- Evaluating the testing results on bugs, errors, database impacts, and usability.
- Preparing the reports on all the activities carried out while testing the sw and reporting to design time.

Software testing life cycle:

It is a process that identifies which test activities to perform and when to execute those test activities



Requirement analysis – identifies whether we can test the requirements or not

Test planning – tool selection, resource planning, deciding roles

Test design – testing team develops the test cases

Test env setup -

Test execution - after fixing the defect, retesting will be performed

Test closure – prepare the test metrics and test closure report.

Why do we develop test cases:

For maintaining consistency in the test case execution

To ensure better test coverage

Test case template:

Test case type – test case can be system, integration or functional test case.

Step number – it is critical field because if tep number 10 is failing, we can report and prioritize working.

Pre condition – it is required changes that must be satisfied by all the test engineers before beginning the test execution process

Test data